

De Registros a Indicadores

Agregaciones y Análisis de Negocio con SQL

INGRESOS TOTALES

\$1.450.000

Los datos transaccionales por sí solos no responden preguntas estratégicas

Miles de registros unidos (JOIN)

[illegible]

¿Cuántas ventas totales tenemos?



¿Cuánto dinero hemos generado?



¿Qué producto lidera los ingresos?

Una consulta JOIN perfecta puede devolver miles de filas. Humanamente, no significan nada sin un proceso de resumen. Ya no necesitamos observar cada registro; necesitamos destilar la información.

El kit de herramientas de comprensión analítica

Símbolo	Función SQL	Pregunta de Negocio	Acción Visual
#	COUNT()	¿Cuántos registros o eventos existen?	
Σ	SUM()	¿Cuánto es el volumen o dinero total?	
$\oplus$	AVG()	¿Cuál es el comportamiento promedio?	
↓	MIN()	¿Cuál es el piso o el peor escenario?	
↑	MAX()	¿Cuál es el techo o el mejor escenario?	



Regla de oro: Siempre utiliza alias de columna (AS total_ventas). El objetivo es generar reportes legibles para humanos, no exportar columnas genéricas llamadas COUNT(*).

Construyendo métricas invisibles en tiempo de ejecución



```
SELECT SUM(cantidad * precio_unitario) AS ingresos_totales  
FROM detalle_ventas;
```

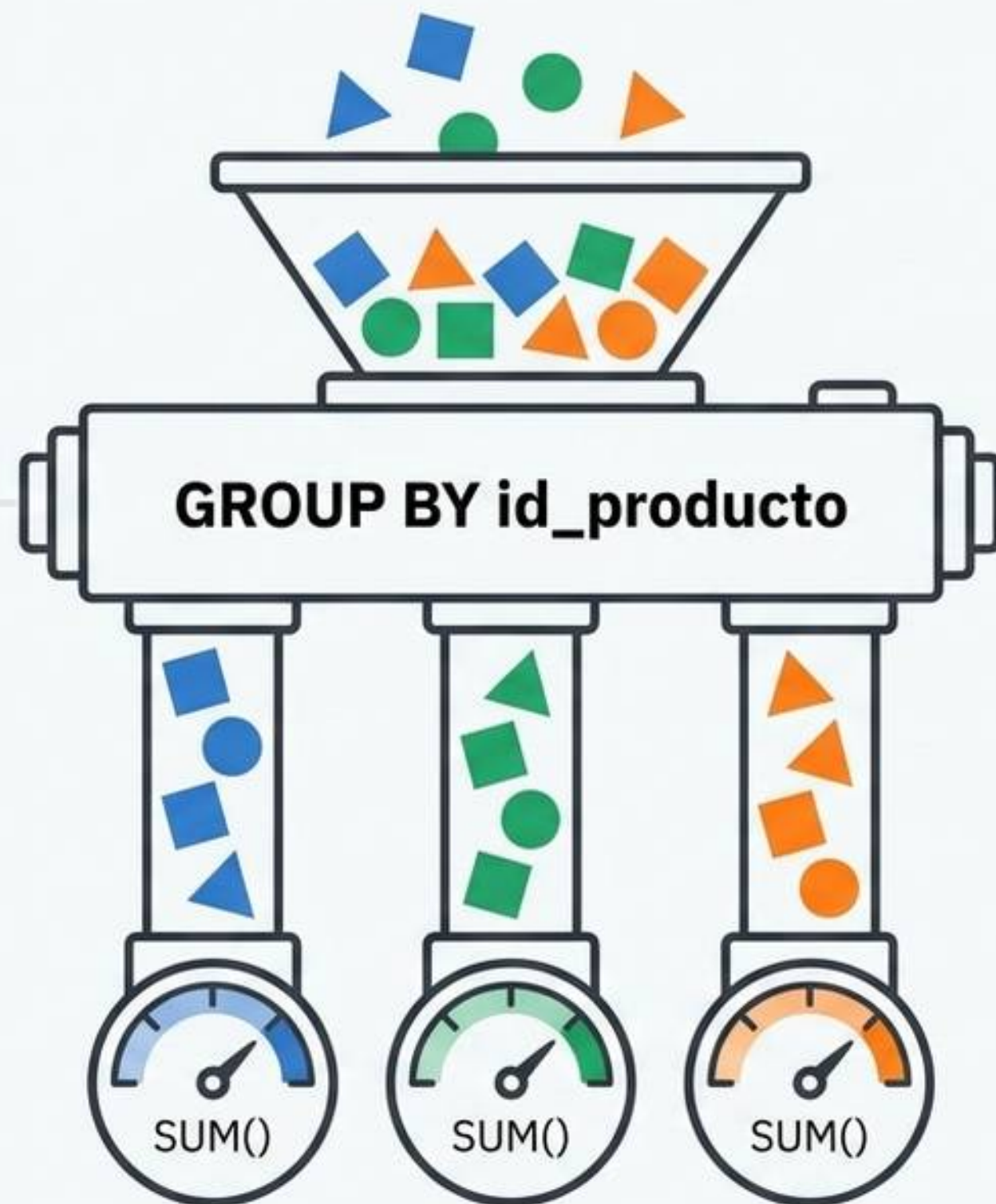
El indicador analítico no está almacenado en la base de datos; **se construye dinámicamente** agregando el producto matemático de las transacciones individuales.

El salto analítico: de totales globales a totales por categoría

Antes (Global):

```
SELECT  
SUM(ingresos)  
FROM ventas;
```

Pregunta:
¿Cuánto vendimos
en total?



Después
(Categorico):

```
SELECT  
id_producto,  
SUM(ingresos)  
FROM ventas  
GROUP BY  
id_producto;
```

Pregunta:
¿Cuánto vendimos
por producto?

Integración relacional: traduciendo IDs a nombres comerciales

detalle_ventas (dv)

Contiene: cantidad * precio

productos (p)

Contiene: nombre_producto



INNER JOIN (id_producto)

```
SELECT p.nombre AS producto,  
       SUM(dv.cantidad * dv.precio_unitario) AS ingresos  
FROM detalle_ventas dv  
INNER JOIN productos p ON dv.id_producto = p.id_producto  
GROUP BY p.nombre;
```

GROUP BY p.nombre

Producto	Ingresos
Notebook	\$1.450.000
Monitor	\$820.000
Mouse	\$375.000

Generación de rankings comerciales con ordenamiento jerárquico



El ordenamiento transforma un simple resumen matemático en una herramienta estratégica de decisión para gerencia (Top Sellers).

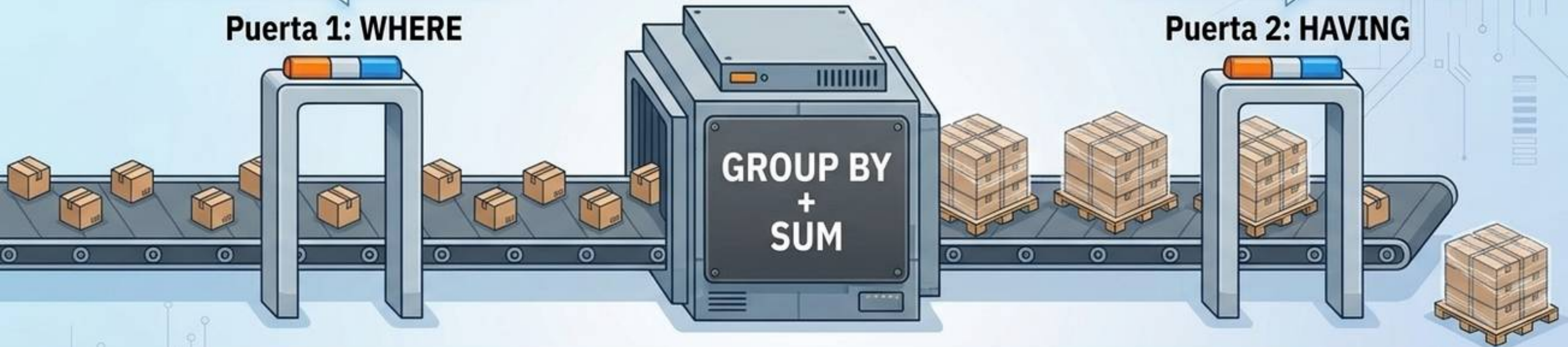
Las dos puertas de peaje en la refinería de datos

Filtra filas individuales
(Ej: descartar ventas canceladas)

Puerta 1: WHERE

Filtra grupos enteros
(Ej: Solo pallets > \$500.000)

Puerta 2: HAVING



Cláusula	Acción	Momento de ejecución
WHERE	Filtra registros individuales crudos	ANTES de agrupar
HAVING	Filtra resultados matemáticos agregados	DESPUÉS de agrupar

Regla de Oro: Si utilizas una función matemática (SUM, COUNT) como condición para filtrar, la única puerta válida es HAVING.

Anatomía y orden cronológico de ejecución de una consulta

```
SELECT p.categoria,  
       SUM(dv.cantidad * dv.precio)  
       AS ingresos  
FROM detalle_ventas dv  
INNER JOIN productos p ON  
       dv.id_producto = p.id_producto  
  
GROUP BY p.categoria  
HAVING SUM(dv.cantidad *  
           dv.precio) > 500000  
  
ORDER BY ingresos DESC;
```

Fase 1: Origen (FROM + JOIN)

-> Recolectar el universo de datos.

Fase 2: Segmentación (GROUP BY)

-> Crear las cubetas categóricas.

Fase 3: Filtrado Agregado (HAVING)

-> Descartar cubetas irrelevantes.

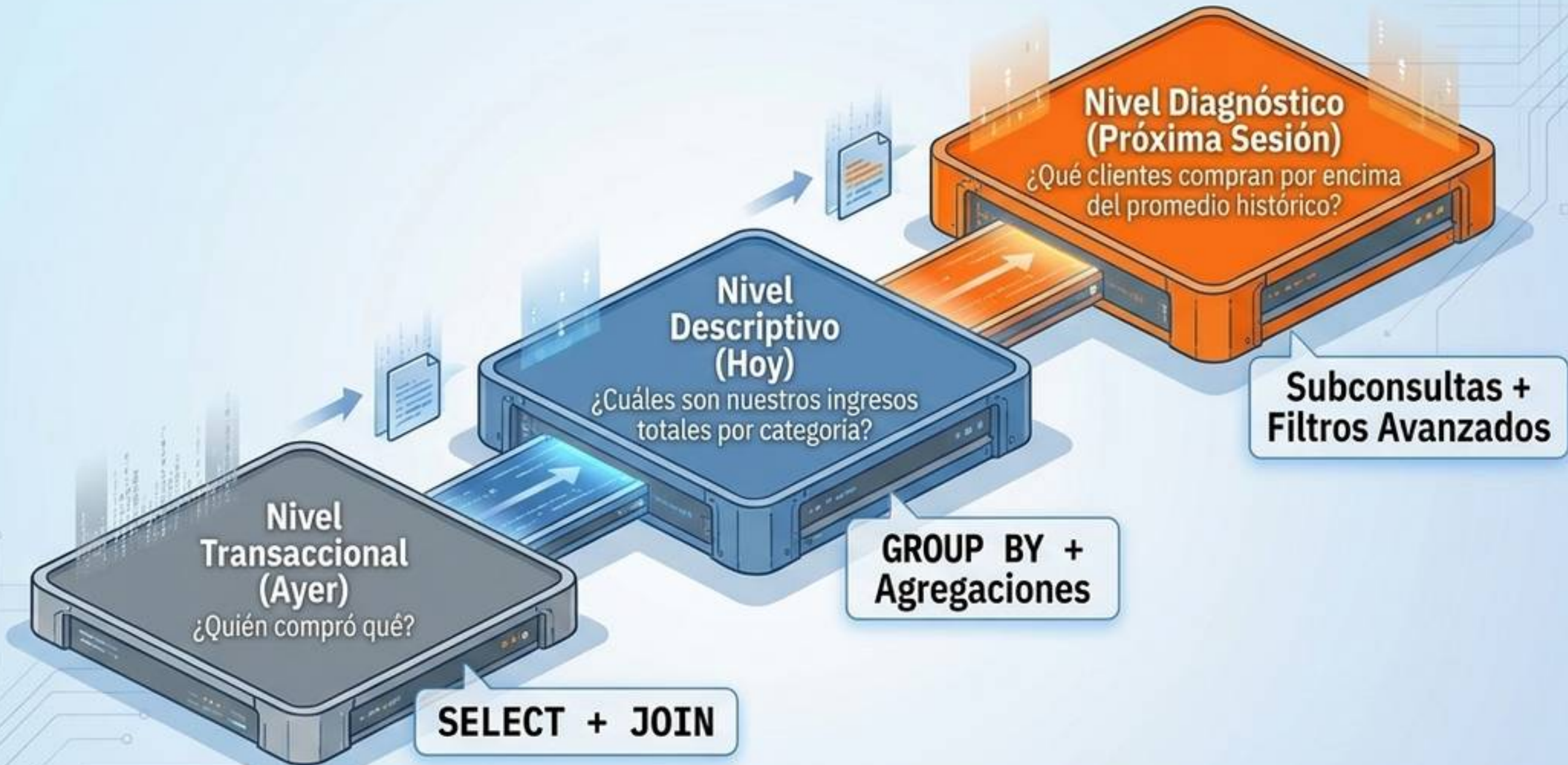
Fase 4: Cálculo (SELECT)

-> Ejecutar sumas matemáticas.

Fase 5: Presentación (ORDER BY)

-> Definir el ranking visual.

De ejecutar código a diagnosticar el negocio



La sintaxis se vuelve más compleja, pero el objetivo final permanece intacto: convertir la niebla de los datos masivos en claridad comercial y estratégica.